

Deep Networks

Regularization: Heuristic Methods

Introduction to Deep Learning · Class 19

Outline

What Is Left to Change

Early Stopping

Averaging Models

Dropout

Applying Noise

Borrowing Other Data

Putting It Together

What Is Left to Change

From the objective to the procedure

Class 18 added a term to the objective and read off the preference the optimiser already had; neither touched the procedure or the data.

Where the bias goes	Mechanism	Class
the objective	$\lambda\ \mathbf{w}\ ^2$, the lasso	18
the optimiser	step size, batch size	18
the architecture	sharing, residual paths	18
the stopping rule	early stopping	today
the number of runs	ensembles, dropout	today
the data	noise, augmentation	today

Everything today is a heuristic: justified by measurement, not by a guarantee.
So everything today is measured.

Notation for today

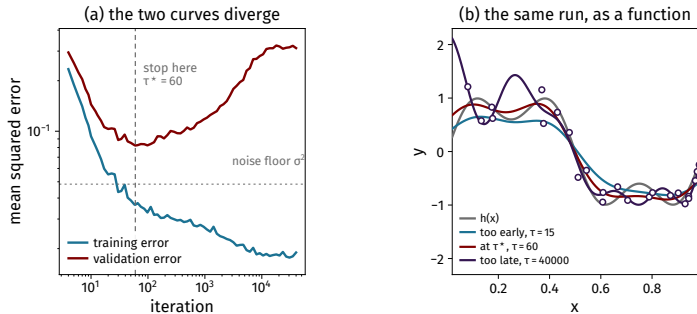
τ	the iteration index	M	members of a committee
η	the step size, as in Class 13	$y_m(\mathbf{x})$	the prediction of member m
α	the weight-decay strength	ρ	the dropout probability
λ_i	eigenvalues of $\mathbf{H}$, as in Class 13	σ_x^2	variance of noise added to inputs

- Bishop writes the dropout *keep* probability and Prince writes the *drop* probability; the two are $1 - \rho$ and ρ
- $h(\mathbf{x})$ is still the regression function of Class 17

Bishop & Bishop (2024), §9.3.1, §9.6; Prince (2023), §9.3.

Early Stopping

The learning curve is a measurement, not a prediction



(a) training error falls throughout; validation error turns at $\tau^* = 60$. (b) the fit before, at, and long after that point: the coarse shape arrives first, the noise afterwards.

Bishop & Bishop (2024), Fig. 9.7; Prince (2023), Fig. 9.6.

Stopping early, in practice

- One hyperparameter: how many steps. Unlike every other hyperparameter, it does **not** need repeated training
- Train once, evaluate on the validation set every T steps, keep the parameters whenever the validation error improves, and return the best set at the end
- So the cost is one run plus the evaluations, against one run *per value* for a penalty coefficient
- In the language of Class 17: increasing τ increases the effective model complexity, so stopping early moves back down the curve rather than changing the model

Prince (2023), §9.3.1; Bishop & Bishop (2024), §9.3.1.

Stopping early is weight decay in disguise

Theorem 1 (Early stopping and weight decay)

On a quadratic error with Hessian eigenvalues λ_i , gradient descent started from $\mathbf{w} = \mathbf{0}$ returns, after τ steps,

$$w_i^{(\tau)} = (1 - (1 - \eta\lambda_i)^\tau) w_i^*, \quad \text{against} \quad \hat{w}_i = \frac{\lambda_i}{\lambda_i + \alpha} w_i^*$$

for weight decay of strength α . Both retain the stiff directions and discard the flat ones, and they change over at the same curvature once $\alpha = 1/(\eta\tau)$.

- So $\eta\tau$ acts as the reciprocal of a regularization coefficient: training longer is regularising less

Bishop & Bishop (2024), §9.3.1, Fig. 9.8; Bishop (1995a).

Where that comes from

- In the eigenbasis the quadratic separates, so each coordinate evolves on its own:
 $w_i^{(\tau)} - w_i^* = (1 - \eta\lambda_i)(w_i^{(\tau-1)} - w_i^*).$
- Starting from $w_i^{(0)} = 0$ and iterating gives $w_i^{(\tau)} - w_i^* = -(1 - \eta\lambda_i)^\tau w_i^*$, which rearranges to the stated filter.
- For small $\eta\lambda_i$ that filter is $\approx 1 - e^{-\eta\tau\lambda_i}$, a function of the single combination $\eta\tau\lambda_i$.
- Weight decay's filter is $\lambda_i/(\lambda_i + \alpha) = u/(1 + u)$ with $u = \lambda_i/\alpha$.
- Both are 0 at $u = 0$, both tend to 1, and both pass their halfway point at $u \approx 1$. Setting $\alpha = 1/(\eta\tau)$ makes the two u 's the same number. □

Directions with $\lambda_i \gg 1/(\eta\tau)$ have been learned; those with $\lambda_i \ll 1/(\eta\tau)$ have not been reached yet. The clock is doing what the penalty would have done.

Where that comes from

- In the eigenbasis the quadratic separates, so each coordinate evolves on its own:
 $w_i^{(\tau)} - w_i^* = (1 - \eta\lambda_i) (w_i^{(\tau-1)} - w_i^*).$
- **Starting from $w_i^{(0)} = 0$ and iterating gives $w_i^{(\tau)} - w_i^* = -(1 - \eta\lambda_i)^\tau w_i^*$, which rearranges to the stated filter.**
- For small $\eta\lambda_i$ that filter is $\approx 1 - e^{-\eta\tau\lambda_i}$, a function of the single combination $\eta\tau\lambda_i$.
- Weight decay's filter is $\lambda_i/(\lambda_i + \alpha) = u/(1 + u)$ with $u = \lambda_i/\alpha$.
- Both are 0 at $u = 0$, both tend to 1, and both pass their halfway point at $u \approx 1$. Setting $\alpha = 1/(\eta\tau)$ makes the two u 's the same number.

□

Directions with $\lambda_i \gg 1/(\eta\tau)$ have been learned; those with $\lambda_i \ll 1/(\eta\tau)$ have not been reached yet. The clock is doing what the penalty would have done.

Where that comes from

- In the eigenbasis the quadratic separates, so each coordinate evolves on its own:
 $w_i^{(\tau)} - w_i^* = (1 - \eta\lambda_i)(w_i^{(\tau-1)} - w_i^*).$
- Starting from $w_i^{(0)} = 0$ and iterating gives $w_i^{(\tau)} - w_i^* = -(1 - \eta\lambda_i)^\tau w_i^*$, which rearranges to the stated filter.
- **For small $\eta\lambda_i$ that filter is $\approx 1 - e^{-\eta\tau\lambda_i}$, a function of the single combination $\eta\tau\lambda_i$.**
- Weight decay's filter is $\lambda_i/(\lambda_i + \alpha) = u/(1 + u)$ with $u = \lambda_i/\alpha$.
- Both are 0 at $u = 0$, both tend to 1, and both pass their halfway point at $u \approx 1$. Setting $\alpha = 1/(\eta\tau)$ makes the two u 's the same number. □

Directions with $\lambda_i \gg 1/(\eta\tau)$ have been learned; those with $\lambda_i \ll 1/(\eta\tau)$ have not been reached yet. The clock is doing what the penalty would have done.

Where that comes from

- In the eigenbasis the quadratic separates, so each coordinate evolves on its own:
 $w_i^{(\tau)} - w_i^* = (1 - \eta\lambda_i) (w_i^{(\tau-1)} - w_i^*).$
- Starting from $w_i^{(0)} = 0$ and iterating gives $w_i^{(\tau)} - w_i^* = -(1 - \eta\lambda_i)^\tau w_i^*$, which rearranges to the stated filter.
- For small $\eta\lambda_i$ that filter is $\approx 1 - e^{-\eta\tau\lambda_i}$, a function of the single combination $\eta\tau\lambda_i$.
- **Weight decay's filter is $\lambda_i/(\lambda_i + \alpha) = u/(1 + u)$ with $u = \lambda_i/\alpha$.**
- Both are 0 at $u = 0$, both tend to 1, and both pass their halfway point at $u \approx 1$. Setting $\alpha = 1/(\eta\tau)$ makes the two u 's the same number. □

Directions with $\lambda_i \gg 1/(\eta\tau)$ have been learned; those with $\lambda_i \ll 1/(\eta\tau)$ have not been reached yet. The clock is doing what the penalty would have done.

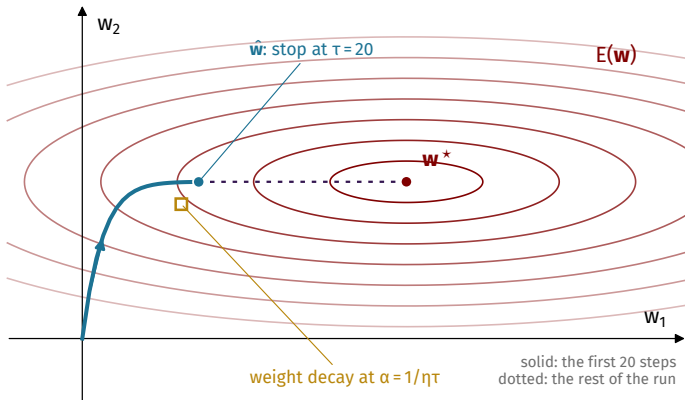
Where that comes from

- In the eigenbasis the quadratic separates, so each coordinate evolves on its own:
 $w_i^{(\tau)} - w_i^* = (1 - \eta\lambda_i) (w_i^{(\tau-1)} - w_i^*).$
- Starting from $w_i^{(0)} = 0$ and iterating gives $w_i^{(\tau)} - w_i^* = -(1 - \eta\lambda_i)^\tau w_i^*$, which rearranges to the stated filter.
- For small $\eta\lambda_i$ that filter is $\approx 1 - e^{-\eta\tau\lambda_i}$, a function of the single combination $\eta\tau\lambda_i$.
- Weight decay's filter is $\lambda_i/(\lambda_i + \alpha) = u/(1 + u)$ with $u = \lambda_i/\alpha$.
- **Both are 0 at $u = 0$, both tend to 1, and both pass their halfway point at $u \approx 1$. Setting $\alpha = 1/(\eta\tau)$ makes the two u 's the same number.**



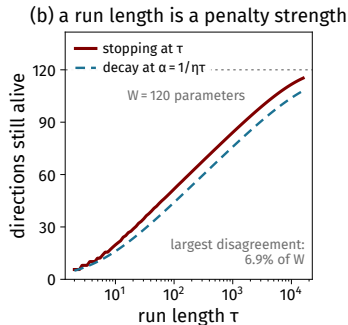
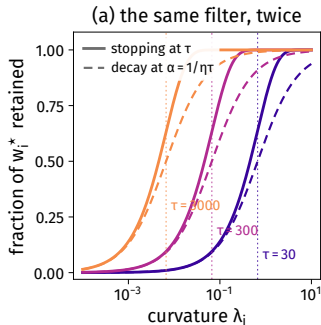
Directions with $\lambda_i \gg 1/(\eta\tau)$ have been learned; those with $\lambda_i \ll 1/(\eta\tau)$ have not been reached yet. The clock is doing what the penalty would have done.

Descent on a quadratic, drawn



Contours of the error, axes on the eigenvectors of the Hessian. From the origin, descent moves first along w_2 , the stiff direction, and only later along w_1 towards $\mathbf{w}^*$. Stopping at $\hat{\mathbf{w}}$ lands beside what a weight-decay penalty at $\alpha = 1/\eta\tau$ would have given. Bishop & Bishop (2024), Fig. 9.8.

The two filters, measured



(a) the two filters at three run lengths and their matched penalties; each pair crosses over together. (b) directions left alive, over a spectrum of 120 eigenvalues: the two never differ by more than 7% of W .

Averaging Models

Committees

$$y_{\text{COM}}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M y_m(\mathbf{x}), \quad p(y | \mathbf{x}) = \frac{1}{M} \sum_{m=1}^M p_m(y | \mathbf{x}).$$

- Rather than select the best of several models, keep all of them and average
- Class 17 showed why this should help: the **variance** term is what differs between models fitted to different samples, and averaging cancels it
- The members must differ. Different initializations, different architectures, or — most simply — different data
- **Bagging**: draw N points from the training set *with replacement*, M times over, and fit one model to each resample

Bishop & Bishop (2024), Eqs. 9.42–9.43; Breiman (1996); Prince (2023), §9.3.2.

What averaging is worth

Theorem 2 (Committee error)

Write $y_m(\mathbf{x}) = h(\mathbf{x}) + \epsilon_m(\mathbf{x})$ and let $E_{AV} = \frac{1}{M} \sum_m \mathbb{E}_{\mathbf{x}}[\epsilon_m^2]$. If the errors have zero mean and are uncorrelated, $\mathbb{E}_{\mathbf{x}}[\epsilon_m \epsilon_l] = 0$ for $m \neq l$, then

$$E_{COM} = \frac{1}{M} E_{AV}.$$

Without any assumption at all, $E_{COM} \leq E_{AV}$.

- The first statement is spectacular and its hypothesis is almost never true: members trained on resamples of one data set make **correlated** errors
- The second needs nothing and is the one to rely on: averaging never hurts

Bishop & Bishop (2024), Eqs. 9.44–9.50 and Exercise 9.15.

Proof of both halves

- **By definition** $E_{\text{COM}} = \mathbb{E}_{\mathbf{x}}[(\frac{1}{M} \sum_m \epsilon_m)^2]$.
- Expanding the square gives $\frac{1}{M^2} \sum_m \mathbb{E}[\epsilon_m^2] + \frac{1}{M^2} \sum_{m \neq l} \mathbb{E}[\epsilon_m \epsilon_l]$.
- If the cross terms vanish, only the first sum survives: $\frac{1}{M^2} \cdot M E_{\text{AV}} = E_{\text{AV}}/M$. □
- For the bound, apply Jensen's inequality to the convex function $t \mapsto t^2$: $(\frac{1}{M} \sum_m \epsilon_m)^2 \leq \frac{1}{M} \sum_m \epsilon_m^2$ pointwise in $\mathbf{x}$.
- Taking $\mathbb{E}_{\mathbf{x}}$ of both sides gives $E_{\text{COM}} \leq E_{\text{AV}}$, with no assumption on the correlations. □

The factor M is a best case that requires independence. The inequality is free.

Proof of both halves

- By definition $E_{\text{COM}} = \mathbb{E}_{\mathbf{x}}[(\frac{1}{M} \sum_m \epsilon_m)^2]$.
- **Expanding the square gives** $\frac{1}{M^2} \sum_m \mathbb{E}[\epsilon_m^2] + \frac{1}{M^2} \sum_{m \neq l} \mathbb{E}[\epsilon_m \epsilon_l]$.
- If the cross terms vanish, only the first sum survives: $\frac{1}{M^2} \cdot M E_{\text{AV}} = E_{\text{AV}}/M$. □
- For the bound, apply Jensen's inequality to the convex function $t \mapsto t^2$: $(\frac{1}{M} \sum_m \epsilon_m)^2 \leq \frac{1}{M} \sum_m \epsilon_m^2$ pointwise in $\mathbf{x}$.
- Taking $\mathbb{E}_{\mathbf{x}}$ of both sides gives $E_{\text{COM}} \leq E_{\text{AV}}$, with no assumption on the correlations. □

The factor M is a best case that requires independence. The inequality is free.

Proof of both halves

- By definition $E_{\text{COM}} = \mathbb{E}_{\mathbf{x}}[(\frac{1}{M} \sum_m \epsilon_m)^2]$.
- Expanding the square gives $\frac{1}{M^2} \sum_m \mathbb{E}[\epsilon_m^2] + \frac{1}{M^2} \sum_{m \neq l} \mathbb{E}[\epsilon_m \epsilon_l]$.
- **If the cross terms vanish, only the first sum survives: $\frac{1}{M^2} \cdot M E_{\text{AV}} = E_{\text{AV}}/M$.** □
- For the bound, apply Jensen's inequality to the convex function $t \mapsto t^2$: $(\frac{1}{M} \sum_m \epsilon_m)^2 \leq \frac{1}{M} \sum_m \epsilon_m^2$ pointwise in $\mathbf{x}$.
- Taking $\mathbb{E}_{\mathbf{x}}$ of both sides gives $E_{\text{COM}} \leq E_{\text{AV}}$, with no assumption on the correlations. □

The factor M is a best case that requires independence. The inequality is free.

Proof of both halves

- By definition $E_{\text{COM}} = \mathbb{E}_{\mathbf{x}}[(\frac{1}{M} \sum_m \epsilon_m)^2]$.
- Expanding the square gives $\frac{1}{M^2} \sum_m \mathbb{E}[\epsilon_m^2] + \frac{1}{M^2} \sum_{m \neq l} \mathbb{E}[\epsilon_m \epsilon_l]$.
- If the cross terms vanish, only the first sum survives: $\frac{1}{M^2} \cdot M E_{\text{AV}} = E_{\text{AV}}/M$. □
- **For the bound, apply Jensen's inequality to the convex function $t \mapsto t^2$:**
 $(\frac{1}{M} \sum_m \epsilon_m)^2 \leq \frac{1}{M} \sum_m \epsilon_m^2$ **pointwise in $\mathbf{x}$.**
- Taking $\mathbb{E}_{\mathbf{x}}$ of both sides gives $E_{\text{COM}} \leq E_{\text{AV}}$, with no assumption on the correlations. □

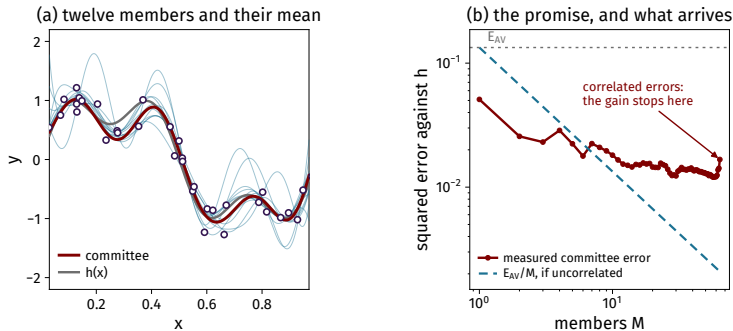
The factor M is a best case that requires independence. The inequality is free.

Proof of both halves

- By definition $E_{\text{COM}} = \mathbb{E}_{\mathbf{x}}[(\frac{1}{M} \sum_m \epsilon_m)^2]$.
- Expanding the square gives $\frac{1}{M^2} \sum_m \mathbb{E}[\epsilon_m^2] + \frac{1}{M^2} \sum_{m \neq l} \mathbb{E}[\epsilon_m \epsilon_l]$.
- If the cross terms vanish, only the first sum survives: $\frac{1}{M^2} \cdot M E_{\text{AV}} = E_{\text{AV}}/M$. □
- For the bound, apply Jensen's inequality to the convex function $t \mapsto t^2$: $(\frac{1}{M} \sum_m \epsilon_m)^2 \leq \frac{1}{M} \sum_m \epsilon_m^2$ pointwise in $\mathbf{x}$.
- Taking $\mathbb{E}_{\mathbf{x}}$ of both sides gives $E_{\text{COM}} \leq E_{\text{AV}}$, with no assumption on the correlations. □

The factor M is a best case that requires independence. The inequality is free.

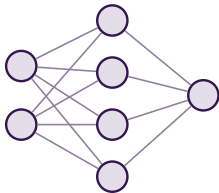
The promise, and what arrives



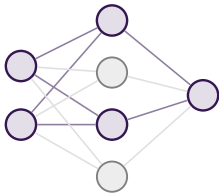
(a) twelve members fitted to bootstrap resamples, and their mean. (b) the committee error against M : it falls by about 4.5 and flattens, where $1/M$ would promise 64. The members' errors are correlated.

Dropout

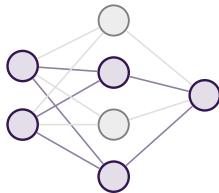
An ensemble you do not have to train



full network



one thinned copy



another

- At every step a fresh random mask deletes each non-output node with probability ρ , and the update is computed on what is left
- With M non-output nodes there are 2^M thinned networks; almost none is ever visited twice, and they **share** their weights

Srivastava et al. (2014); Bishop & Bishop (2024), §9.6.1, Fig. 9.17; Prince (2023), Fig. 9.8.

How to predict with 2^M networks

Proposition 3 (The weight-scaling inference rule)

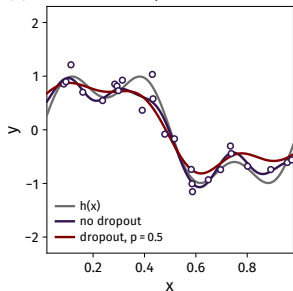
A node kept with probability $1 - \rho$ contributes, in expectation over masks, $(1 - \rho)$ times what it contributes when always present. Multiplying its outgoing weights by $(1 - \rho)$ at test time therefore matches the expected input to the next node, and lets one deterministic pass stand in for the average over masks.

- The exact rule would be $p(y | \mathbf{x}) = \sum_{\mathbf{R}} p(\mathbf{R}) p(y | \mathbf{x}, \mathbf{R})$, a sum over exponentially many masks
- **Monte Carlo dropout** instead samples 10 or 20 masks and averages, which also gives a spread and so an uncertainty
- Typical rates: $\rho \approx 0.5$ for hidden nodes, $\rho \approx 0.2$ for inputs

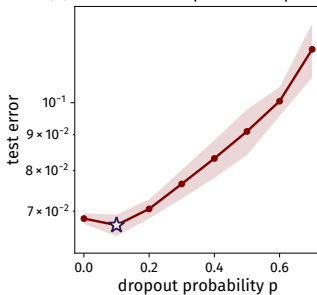
Bishop & Bishop (2024), Eq. 9.51; Prince (2023), §9.3.3.

Dropout, measured

(a) the same model, thinned while training



(b) the best rate is problem-dependent



(a) the same model with and without dropout: the dropout fit is smoother where the data are thin. (b) test error against the rate, six runs each. The best rate here is 0.1, not the usual 0.5: with 44 features there is little redundancy to spare.

Why removing a unit removes a kink

- Suppose three units conspire: one raises the slope, the next lowers it, the third restores it. The net effect is a local wiggle between two training points
- That wiggle costs nothing on the training set, so no amount of further training removes it
- Delete one of the three — which is what dropout does — and the compensation fails, the output changes over a whole half-space, and the loss notices
- The next gradient step then repairs it, and over many steps the conspiracy is dismantled

Dropout reaches structure that the loss alone cannot see, because it changes which parameters are asked to explain the data.

Prince (2023), §9.3.3 and Fig. 9.9.

Applying Noise

Noise on the inputs is a penalty

Theorem 4 (Input noise and weight decay)

Let the model be linear, $\hat{y} = \mathbf{x}^\top \mathbf{w}$, and add noise ϵ with mean $\mathbf{0}$ and covariance $\sigma_x^2 \mathbf{I}$ to every input. Then

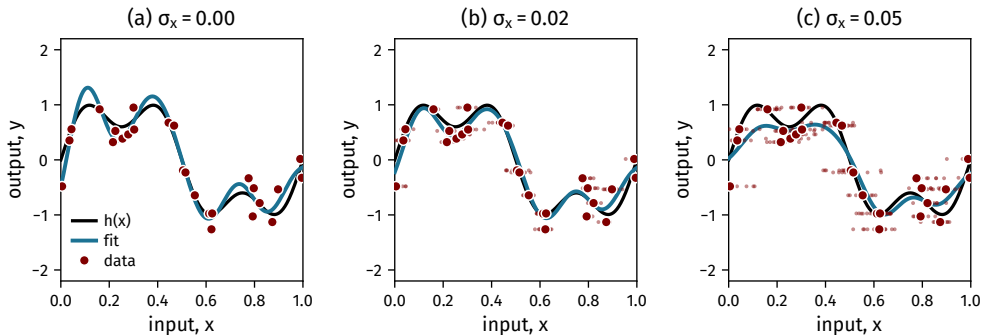
$$\mathbb{E}_\epsilon [((\mathbf{x} + \epsilon)^\top \mathbf{w} - Y)^2] = (\mathbf{x}^\top \mathbf{w} - Y)^2 + \sigma_x^2 \|\mathbf{w}\|^2,$$

so training on noisy inputs is exactly weight decay with $\lambda = \sigma_x^2$. For a nonlinear model the same expansion leaves a penalty on $\partial \hat{y} / \partial \mathbf{x}$ instead.

- The proof is one line: expand the square and use $\mathbb{E}[\epsilon] = \mathbf{0}$ and $\mathbb{E}[\epsilon \epsilon^\top] = \sigma_x^2 \mathbf{I}$
- The nonlinear version penalises **sensitivity of the output to the input**, which is what smoothness means

Bishop (1995b); Prince (2023), §9.3.4.

The equivalence, checked



Noise of standard deviation σ_x added to the batch inputs at every step of SGD, at three levels; the small dots are ten samples of the perturbed data. More noise gives a smoother fit. For a model linear in its inputs the equivalence is exact: trained with noise and solved with ridge at $\lambda = \sigma_x^2$, the weights differ by under 2% at every level tried. Prince (2023), Fig. 9.10.

Three places to put noise

Noise on	Effect	Known as
the inputs	penalises $\partial \hat{y} / \partial \mathbf{x}$	—
the inputs, worst case	robustness to the worst perturbation	adversarial training
the weights	prefers wide flat minima	—
the labels	discourages certainty	label smoothing
the activations	multiplicative Bernoulli noise	dropout

- Label smoothing assumes a fraction ρ of the labels are wrong and belong equally to the other classes, which stops the pre-softmax activations running away
- Dropout is the same idea applied to activations, which is why it sits in this list as well as the previous section

Prince (2023), §9.3.4.

Borrowing Other Data

When the data set is the constraint

transfer
pre-train elsewhere,
refit the head

needs a related
labelled set

multi-task
one trunk,
several outputs

needs several
label types

self-supervised
hide part of
the input,
predict it

needs no
labels at all

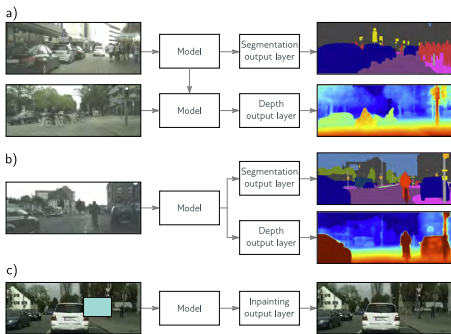
augmentation
transform the input,
keep the label

needs a known
invariance

- All four are ways of getting more constraint per parameter without collecting more labelled examples of the actual task
- Augmentation is the invariance route of Class 18, taken through the data rather than the architecture

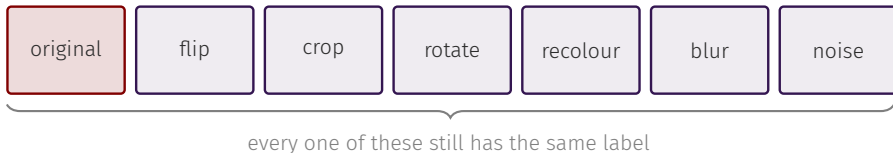
Prince (2023), §9.3.6–9.3.8; Bishop & Bishop (2024), §9.1.3.

The three routes, drawn



(a) transfer learning: train on a secondary task with plentiful labels, then replace the head. (b) multi-task learning: one trunk, several outputs. (c) generative self-supervision: hide part of the input and train the network to restore it, so no labels are needed at all. Prince (2023), Fig. 9.12.

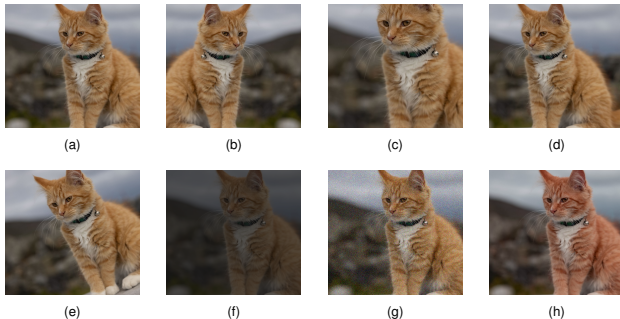
Augmentation



- Each transformation must be one the label is genuinely invariant to; a vertical flip is fine for a bird and wrong for a digit
- For text: substitute synonyms, or translate out and back. For audio: change the gain of frequency bands
- The model is being taught what to **ignore**, which is a statement about the task and not about the data set

Prince (2023), §9.3.8, Fig. 9.13; Bishop & Bishop (2024), Fig. 9.1.

Augmentation, on an image



(a) the original image, then (b) horizontal inversion, (c) scaling, (d) translation, (e) rotation, (f) brightness and contrast, (g) additive noise, (h) colour shift. Every one still has the same label. Bishop & Bishop (2024), Fig. 9.1.

The Bayesian view: an infinite committee

Definition 5 (Bayesian prediction)

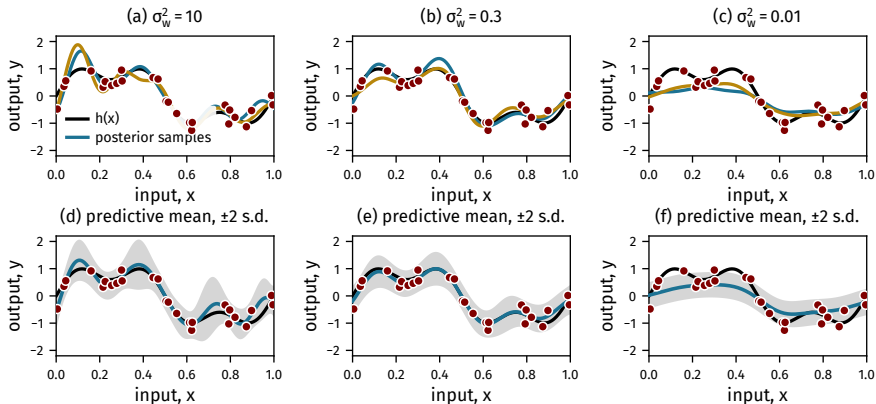
Instead of one parameter vector, keep a posterior over all of them,

$$p(\mathbf{w} \mid \mathcal{D}) \propto \left[\prod_n p(Y_n \mid \mathbf{x}_n, \mathbf{w}) \right] p(\mathbf{w}), \quad p(y \mid \mathbf{x}, \mathcal{D}) = \int p(y \mid \mathbf{x}, \mathbf{w}) p(\mathbf{w} \mid \mathcal{D}) d\mathbf{w}.$$

- This is a committee with one member per parameter vector, weighted by how well it fits and how plausible it was
- Class 18's penalty was the same prior $p(\mathbf{w})$, used to pick the single most probable member instead of averaging
- For a network the integral is intractable, so every practical method approximates it — and Monte Carlo dropout is one such approximation

Prince (2023), §9.3.5, Eqs. 9.11–9.12.

The posterior, drawn



The bump model with a Gaussian prior, where the posterior is exact. (a–c) two parameter vectors drawn from it under priors of three variances: a tighter prior gives smaller parameters and smoother functions. (d–f) the average over all parameter vectors, with two standard deviations shaded: the band narrows from 1.08 to 0.49.

Prince (2023), Fig. 9.11.

Putting It Together

The whole toolbox, across three classes

Method	Acts on	Costs	Class
weight decay, lasso	the objective	one run per λ	18
step size, batch size	the optimiser	nothing extra	18
sharing, residual paths	the architecture	must know the symmetry	18
early stopping	the stopping rule	one run, plus validation	19
ensembling, bagging	the number of models	M trainings, M inferences	19
dropout	the activations	slower convergence	19
input, weight, label noise	the data	one hyperparameter each	19
augmentation	the data	more compute per epoch	19
transfer, self-supervision	other data	a second training stage	19

Every row buys the same thing — less variance — and each pays in a different currency: compute, hyperparameters, or knowledge about the task.

Takeaways

1. Early stopping is the cheapest regularizer there is: one run, and the hyperparameter is read off a curve you were plotting anyway
2. On a quadratic it is weight decay with $\alpha = 1/(\eta\tau)$, so training longer regularises less (Theorem 1)
3. Averaging M models divides the error by M only if their errors are uncorrelated; what always holds is $E_{\text{COM}} \leq E_{\text{AV}}$ (Theorem 2)
4. In practice the members are correlated, and the measured gain was a factor of 4.5 where $1/M$ promised 64
5. Dropout approximates an ensemble over 2^M weight-sharing networks; at test time one scaled pass stands in for it (Proposition 3)
6. Noise on the inputs is weight decay exactly, for a linear model, and a penalty on $\partial\hat{y}/\partial\mathbf{x}$ in general (Theorem 4)
7. These are heuristics. Every one of the numbers above was measured on one small problem, and the defaults that circulate were calibrated on much larger ones

Class 20 turns from how to train a network to what to build: the tutorials, and then convolutional networks.

References i

1. C. M. Bishop & H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. §9.3.1 (Figs. 9.7–9.8); §9.6 (Eqs. 9.42–9.51, Fig. 9.17); Exercises 9.6, 9.14, 9.15.
2. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. §9.3 (Eqs. 9.11–9.12, Figs. 9.5–9.13).
3. C. M. Bishop, “Regularization and complexity control in feed-forward networks”, ICANN 1995.
4. C. M. Bishop, “Training with noise is equivalent to Tikhonov regularization”, *Neural Computation* 7(1), 1995.
5. L. Breiman, “Bagging predictors”, *Machine Learning* 24(2), 1996.
6. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever & R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting”, *JMLR* 15, 2014.
7. Y. Gal & Z. Ghahramani, “Dropout as a Bayesian approximation”, ICML 2016.
8. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens & Z. Wojna, “Rethinking the Inception architecture”, CVPR 2016. (label smoothing)
9. I. J. Goodfellow, J. Shlens & C. Szegedy, “Explaining and harnessing adversarial examples”, ICLR 2015.
10. Y. Freund & R. E. Schapire, “Experiments with a new boosting algorithm”, ICML 1996.

Questions?